UNIVERSITY OF OTTAWA — FACULTY OF ENGINEERING
Course: BMG 5111 / ELG 6131 / DTI 6302 — eHealth, mHealth & Telemedicine
Student/Team: Suganthan M. & Collaborator (Group 12 — *Hackers-Healers-1*)
Meeting Purpose: Pre-Meeting Technical Audit of Downloaded Mobile App Codebase
Meeting Time: Today, October 16, 2026 @ 8:00 PM EST

e-Hospital Mobile App Review & Strategy Brief

Technical Evaluation of the Summer 2026 Flutter/FastAPI App & High-Yield Questions for Tonight's 8:00 PM Meeting

1. Codebase Identification & Executive Summary

Downloaded Repository: 2026-summer-11-AI-automated-queries-based-on-the-db-schema-and-do-analysis-with-UI-main
Origin & Author: Developed by Yuyang Wei (Student #300484288) for the Summer 2026 cohort of DTI 6302. It refactors and expands an earlier Winter 2026 project (2026-winter-Unified-Data-Analysis-System...).
What the System Is: A hybrid patient-facing mobile solution comprising a **Flutter mobile client** (targeting iOS/HealthKit) and a companion **Python FastAPI AI backend** that orchestrates patient queries, LangGraph-style LLM workflows (Ollama / Gemini), medical report OCR parsing, and integrates with the course's shared AWS App Runner database (https://aetab8pjmb.us-east-1.awsapprunner.com).

2. System Architecture & Component Mapping

Layer	Component & Tech Stack	Key Responsibilities & Data Handlers
Mobile App (Client)	**Flutter (Dart)** • Material 3 • HealthKit Native Bridge • fl_chart & SharedPreferences	• Patient authentication & session persistence. • Apple Watch vitals ingestion (Heart Rate, Blood Pressure, Steps, Sleep). • Patient AI assistant chat & urgent care check-in UI. • Local notifications for blood pressure adherence alerts.
AI Backend (Microservice)	**FastAPI (Python)** • Port 8000 (src.backend.main:app) • LangChain / LangGraph • Tesseract OCR & Poppler	• Aggregates patient longitudinal context from AWS MySQL. • Multi-turn AI chat orchestration (Ollama gemma2:2b / Gemini). • Document parsing: OCR for lab reports, scanned PDFs, images. • Alert triage service (3-hour sustained hypertension window).
Cloud DB (Shared Core)	**AWS App Runner MySQL** • 69 Relational Tables • REST Table & SQL Endpoints	• Stores patient demographics (patients_registration), vitals history, ECG, lab results, and diagnostic predictions (diabetes, stroke, heart disease). • Accessible via GET /table/{table} and POST /sql/select.

3. Deep-Dive: Core Features Implemented

- **Authentication & Session:** Authenticates against eHospital user tables via /login. Supports multiple roles (Patient, Doctor, Clinic, Admin). Stores active patient_id in SharedPreferences.
- **Wearable Vitals Sync:** Bridges native Apple HealthKit events into Flutter for resting heart rate, active calories, sleep intervals, and systolic/diastolic blood pressure.
- **HealthKit Alert Engine:** Analyzes blood pressure across a 3-hour moving window. Evaluates patient baseline and active antihypertensive medication context before triggering supportive local push notifications.
- **Medical Report Interpreter:** Upload pipeline for text, JSON, PDFs, and scanned photos. Uses Tesseract OCR and PyPDF to extract biomarker values and translate abnormal results into plain patient language.
- **Urgent Care & Clinic Workflow:** Ambulatory check-in module calculating estimated wait times, collecting patient feedback, and maintaining a clinical queue status.

4. Critical Codebase Bugs, Bottlenecks & Audit Findings

CRITICAL BUG FOUND — Backend Startup Crash:
In src/backend/api/assistant.py (line 24):
from src.backend.services.assistant.workflows.health_alert_analysis import analyze_health_alert_event
The file health_alert_analysis.py is **completely missing** from the downloaded repository! Starting the backend raises an immediate ModuleNotFoundError. This must be shimmed or recovered from an unmerged branch.

Additional Architectural Weaknesses Discovered:

- **N+1 Serial HTTP Queries & In-Memory Filtering:** In patient_context_service.py, building patient context executes **10 sequential HTTP requests** to AWS App Runner. Crucially, it hits /table/{table} without server-side filtering—downloading entire database tables into Python memory and filtering by patient_id client-side. This creates severe 3–8 second latency bottlenecks.
- **Synchronous LLM Blocking:** In model_client.py, _build_chat_model().invoke(...) is synchronous and invoked directly within async FastAPI routes, freezing the server event loop during LLM token generation.
- **iOS-Only HealthKit Implementation (Android Gap):** The native sensor observer queries are written entirely in Swift (AppDelegate.swift, 740+ lines). The Android folder contains only the empty default Flutter template with **zero Health Connect or Wear OS support**.
- **Native Binary Dependencies:** Medical Report OCR strictly requires local host installation of C++ system binaries (tesseract-ocr and poppler-utils), complicating deployment.
- **Unsecured Cloud Endpoints & Storage:** The AWS App Runner eHospital database has no bearer token / API key requirement, allowing unauthenticated read/write access to all 69 tables. Flutter stores sessions in unencrypted SharedPreferences.

5. Alignment & Synergy with Group 12 (Hackers-Healers-1)

There is a **natural, symbiotic division of labor** between the Summer Mobile App and Group 12's Fall 2026 Proposal (*'e-Hospital AI-Twin: An Agentic Multi-Model Clinical Reasoning Platform'*):

Dimension	Summer Mobile App (Patient-Facing)	Group 12 Hackers-Healers-1 (Clinician-Facing)
Target User	Patient / Individual at Home	Physician, Nurse Practitioner, Clinic Staff
Primary UI	Flutter Mobile App (iOS / Android)	React 18 / Vite Responsive Web Dashboard
Core Capabilities	• Apple Watch vitals sync • Medication tracking & symptom logging • Patient-friendly lab report interpretation • Urgent care check-in	• **Ambient Voice Scribe** (SOAP note generation) • **3D Interactive Patient Digital Twin** (WebGL) • Canadian Guideline Risk Scorer (Hypertension/Diabetes) • Automated OHIP Billing & Specialist Referral Fillers
Data Integration	Writes home vitals & logs to eHospital MySQL	Reads patient vitals into Digital Twin; logs consult notes

6. High-Yield Questions for Tonight's Meeting (8:00 PM)

A. Project Scope & Architecture Boundaries:

- **Q1:** *'Do you want Group 12 to directly adopt and maintain this Flutter codebase as our patient mobile client, or should we treat it as an architectural benchmark while delivering our primary web-based Ambient Scribe and 3D Digital Twin?'*
- **Q2:** *'The summer project introduced an intermediary FastAPI backend. Should we merge our Ambient Voice Scribe and Multi-Agent services into this existing FastAPI service, or maintain microservice boundaries?'*

B. Mobile Platform Constraints (iOS HealthKit vs. Android Gap):

- **Q3:** *'The mobile app's native wearable ingestion is implemented strictly in Swift for iOS HealthKit (AppDelegate.swift), with zero Android Health Connect code. For students or evaluators using Android devices or emulators, are we expected to build the Android Health Connect bridge, or should we rely on the in-app synthetic event injectors (injectTestBloodPressure) for testing and grading?'*
- **Q4:** *'Real-time background delivery for Apple HealthKit requires paid Apple Developer team provisioning and capabilities in Xcode. Does the faculty provide a shared Apple Developer team profile for device testing, or is the app evaluated via tethered foreground execution?'*

C. Database Concurrency & Demo Collisions:

- **Q5:** *'The summer seed script specifically configured Patient IDs 391 and 393 on the live AWS MySQL instance. Are other teams actively using these patient IDs, and should our team partition dedicated patient IDs (e.g., 1200+ for Group 12) to avoid race conditions or overwriting other groups' data during evaluations?'*
- **Q6:** *'We observed significant latency in patient context fetching due to unindexed sequential queries pulling entire tables. May we refactor the data access layer to use direct parameterized POST /sql/select queries with WHERE clauses to reduce latency from ~6s to <400ms?'*

D. Clinical Safety, Privacy & Ethical Compliance (PHIPA / PIPEDA):

- **Q7:** *'In the mobile app's AI assistant, what clinical guardrails are expected for safety? For instance, do you expect deterministic rule-based pre-validation before LLM generation (e.g. flagging critical potassium or troponin values), or is prompt-based system steering with a disclaimer considered sufficient for the rubric?'*
- **Q8:** *'The mobile backend supports both local Ollama and Google Gemini cloud. For our final grading and live demonstration, is cloud LLM usage allowed with synthetic data, or does the course strictly require 100% on-premise local inference to fulfill Ontario PHIPA / PIPEDA data sovereignty standards?'*

E. Telemedicine Integration & Deliverable Weighting:

- **Q9:** *'We noticed the summer app includes an Urgent Care check-in screen, but lacks a clinician-facing triage portal. Since our Group 12 platform already features a clinic dashboard with automated triage, would completing this closed-loop workflow—allowing a patient to check in on mobile and appear instantly in our clinic triage queue—be a valuable deliverable for our project?'*
- **Q10:** *'For our Milestone 3 Midterm Prototype (Nov 4 - 20%), what specific user journey will you and the TAs test during the live presentation? Will you test the mobile app end-to-end, or focus primarily on the core innovations proposed by Group 12 (Ambient Voice Scribe & 3D Digital Twin)?'*

7. Strategic Gameplan for Tonight's Meeting

1. Pitch the 'Two-Tier Ecosystem': Propose positioning the Summer Flutter app as the **Patient Telemedicine Client** (capturing vitals, symptoms, and report OCR) and Group 12's *Hackers-Healers-1* platform as the **Physician Clinical Command Center** (ambient voice scribing, 3D anatomical twin, and OHIP billing). This positions your team as systems integrators rather than merely rebuilding existing screens.
2. Offer a Quick Performance Fix: Mention to the professor that you analyzed their context-fetching pipeline and identified the N+1 query loop. Offer to contribute an optimized parameterized query layer using POST /sql/select that reduces context latency from ~6s down to <400ms.
3. Clarify the Missing Workflow File: Inquire gently about health_alert_analysis.py. This demonstrates that you conducted a line-by-line static code analysis and understand the dependency tree.
4. Lock in Milestone 3 (Nov 4) Deliverables: Ensure the professor clarifies whether live Apple Watch syncing is strictly required for the midterm or if simulated vitals are acceptable.

8. Pre-Meeting Codebase Quick Reference & Run Commands

Target Component	Terminal Execution Command (PowerShell / Bash)
Backend Health Check	curl http://127.0.0.1:8000/report-interpreter/health
Start AI Backend	conda run -n langgraph uvicorn src.backend.main:app --reload --host 0.0.0.0 --port 8000
Flutter Run (Backend Mode)	flutter run --dart-define=AI_PROVIDER=backend --dart-define=BACKEND_BASE_URL=http://127.0.0.1:8000
Seed Demo Patient Data	python src/datasets/seed_llm_health_scenarios.py --refresh-recent (Seeds Patients 391 & 393)
Inspect Patient Context	GET http://127.0.0.1:8000/assistant/patients/391/context
Verify OCR Stack	python -c "import pytesseract, pdf2image; print('OCR Stack Ready')"